

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Петербургский государственный университет путей сообщения
Императора Александра I»
(ФГБОУ ВО ПГУПС)**

Факультет «Автоматизация и интеллектуальные технологии»

Кафедра «Информатика и информационная безопасность»

Специальность 10.05.03 «Информационная безопасность автоматизированных систем»

Специализация «Безопасность автоматизированных систем на железнодорожном транспорте»

КУРСОВАЯ РАБОТА

по дисциплине

«Разработка защищенных приложений под ОС семейства Unix»

на тему: «Практическое применение системных функций Unix-подобной ОС при создании
защищенного программного обеспечения»

Форма обучения – очная

Выполнил обучающийся

Курс 4

Группа КИБ-312

Б.Г. Народицкий

подпись, дата

Руководитель

А. И. Загайнов

подпись, дата

Санкт-Петербург
2026

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Петербургский государственный университет путей сообщения
Императора Александра I»
(ФГБОУ ВО ПГУПС)**

Факультет «Автоматизация и интеллектуальные технологии»

Кафедра «Информатика и информационная безопасность»

Специальность 10.05.03 «Информационная безопасность автоматизированных систем»

Специализация «Безопасность автоматизированных систем на железнодорожном транспорте»

**Задание на выполнение курсовой работы
по дисциплине**

«Разработка защищенных приложений под ОС семейства Unix»

Народицкий Борис Геннадьевич

Тема: «Практическое применение системных функций Unix-подобной ОС при создании защищенного программного обеспечения»

Срок сдачи обучающимся законченной работы: декабря 2026 года

Содержание пояснительной записки:

- задание;
- содержание;
- получение перечня файлов, входящих в каталог;
- применение неименованных каналов для связи между родственными процессами, исполняющими разные программы;
- применение именованных каналов для связи между процессами;
- мультиплексирование сообщений;
- многопоточность, синхронизация потоков;
- применение потоковых сокетов для организации взаимодействия процессов через сеть
- заключение;
- список использованных источников.

Дата выдачи задания: сентября 2026 года

Руководитель: _____ А.И. Загайнов

Задание принял к исполнению: _____ Б.Г. Народицкий

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Петербургский государственный университет путей сообщения
Императора Александра I»
(ФГБОУ ВО ПГУПС)**

Факультет «Автоматизация и интеллектуальные технологии»

Кафедра «Информатика и информационная безопасность»

Специальность 10.05.03 «Информационная безопасность автоматизированных систем»

Специализация «Безопасность автоматизированных систем на железнодорожном транспорте»

Календарный план выполнения и защиты курсовой работы

по дисциплине

«Разработка защищенных приложений под ОС семейства Unix»

Народицкий Борис Геннадьевич

Тема: «Практическое применение системных функций Unix-подобной ОС при создании защищенного программного обеспечения»

№ п/п	Наименование этапов	Планируемая дата	Фактическая дата	Подпись руководителя	Примечание
1	Выдача задания				
2	Разработка программного обеспечения				
3	Сдача курсовой работы на первую проверку				
4	Допуск к защите курсовой работы				
5	Защита курсовой работы	в период промежуточной аттестации, согласно установленному расписанию			

Руководитель _____

(подпись)

« ____ » _____ 2026 г.

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
 Федеральное государственное бюджетное образовательное учреждение
 высшего образования
**«Петербургский государственный университет путей сообщения
 Императора Александра I»
 (ФГБОУ ВО ПГУПС)**

Факультет «Автоматизация и интеллектуальные технологии»

Кафедра «Информатика и информационная безопасность»

Специальность 10.05.03 «Информационная безопасность автоматизированных систем»

Специализация «Безопасность автоматизированных систем на железнодорожном транспорте»

Оценочный лист на курсовую работу

по дисциплине

«Разработка защищенных приложений под ОС семейства Unix»

Народицкий Борис Геннадьевич

Тема: «Практическое применение системных функций Unix-подобной ОС при создании защищенного программного обеспечения»

Оценка курсовой работы

№ п/п	Материалы необходимые для оценки знаний, умений и навыков	Показатель оценивания	Критерии оценивания	Шкала оценивания	Получен ные баллы
1	Выполнение задания по пунктам 2.1 – 2.6, перечисленным в разделе 3 методических рекомендаций	Качество выполнения задания по пункту, в том числе соблюдение установленных сроков	Задание выполнено без замечаний	7	
			Задание в целом выполнено, но имеется один из следующих недостатков: • замечания по программной реализации; • предъявление результатов с опозданием на две недели и более	6	
			Задание в целом выполнено, но имеется два недостатка одновременно: • замечания по программной реализации;	5	

№ п/п	Материалы необходимые для оценки знаний, умений и навыков	Показатель оценивания	Критерии оценивания	Шкала оценивания	Получен ные баллы
			предъявление результатов с опозданием на две недели и более		
		Максимальное количество баллов за выполнение шести пунктов задания		42	
Итого максимальное количество баллов по п. 1				42	
2	Пояснительная записка	Полнота и корректность теоретической части	Теоретическая часть достаточно полно раскрывает тему работы, ошибок и недоработок не выявлено	18	
			Имеется незначительное число ошибок/недоработок	16	
			Количество ошибок/недоработок снижает качество работы в целом	14	
		Оформление пояснительной записки и грамотность изложения материала ¹	Полное соответствие требованиям ГОСТ, отсутствие грамматических ошибок	10	
			Соответствие требованиям ГОСТ	8	
Использование современных средств разработки	В оформлении имеются незначительные нарушения требований ГОСТ, имеется сравнительно небольшое число грамматических ошибок				
Итого максимальное количество баллов по п. 2				28	
ИТОГО максимальное количество баллов				70	
Допуск к защите				≥ 45	

¹ При наличии значительного количества нарушений требований ГОСТ, небрежном оформлении, неграмотном изложении пояснительная записка возвращается на доработку и не оценивается.

Защита работы	– получены полные ответы на вопросы – 25-30 баллов; – получены достаточно полные ответы на вопросы – 21-24 баллов; – получены неполные ответы на вопросы или часть вопросов – 11-20 баллов; не получены ответы на вопросы или вопросы не раскрыты – 0 баллов.	30	
Итоговая оценка	«Отлично» - 86-100 баллов «Хорошо» - 75-85 баллов «Удовлетворительно» - 60-74 баллов «Неудовлетворительно» - менее 59 баллов (вкл.)		

Заключение: рецензируемая курсовая работа соответствует требованиям основной образовательной программы специальности 10.05.03 «Информационная безопасность автоматизированных систем» (специализация «Безопасность автоматизированных систем на железнодорожном транспорте»).

Итоговая оценка –

Руководитель

_____ А.И. Загайнов
(подпись)

« ____ » _____ 2026 г.

ОГЛАВЛЕНИЕ

...

ВВЕДЕНИЕ

...

1. Получение перечня файлов, входящих в каталог

В задании требуется создать программное обеспечение, которое предоставляет список файлов, входящих в каталог.

В Unix/Linux каталог — это специальный файл, который не хранит пользовательские данные, а содержит записи о файлах (directory entries). Каждая запись ставит в соответствие имя файла и номер inode (в классических файловых системах). Сами метаданные файла (права, владелец, размер, времена) хранятся в inode, а не в записи каталога.

Основные особенности

Каталог имеет собственные атрибуты: права доступа, владельца, группу, размер, временные метки. Их можно получить через `stat()`.

Каталог нельзя читать или записывать обычными `read()/write()`. Для работы с ним используются специализированные функции: `opendir()`, `readdir()`, `closedir()`.

Записи каталога не упорядочены. Порядок обхода зависит от файловой системы и не гарантируется.

В каждом каталоге всегда присутствуют две служебные записи:

. — ссылка на сам каталог;

.. — ссылка на родительский каталог.

Функция `readdir()` возвращает указатель на структуру `struct dirent`, содержащую:

`d_ino` — номер inode;

`d_name` — имя файла (без пути);

`d_type` — тип файла (в Linux; может быть `DT_UNKNOWN`).

`readdir()` не возвращает полный путь. Чтобы получить атрибуты файла, необходимо самостоятельно сформировать путь (например, каталог/имя) и вызвать `stat()` или `lstat()`.

`stat()` следует по символической ссылке и возвращает атрибуты целевого файла.

`lstat()` возвращает атрибуты самой ссылки.

Для получения имён владельца и группы используются `getpwuid()` и `getgrgid()` на основе `st_uid` и `st_gid` из `struct stat`.

Для преобразования времени изменения в читаемый вид применяются `ctime()` или `strftime()`.

Права доступа к каталогу

Права на каталог имеют особый смысл:

`r` (`read`) — разрешает чтение списка имён (получение записей);

`w` (`write`) — разрешает создание, удаление и переименование записей;

x (execute) — разрешает доступ к файлам и подкаталогам внутри каталога (поиск по имени, переход).

Без права x нельзя обратиться к файлу по имени, даже если известны его права.

Обход каталога

Типичный алгоритм:

Открыть каталог через opendir().

В цикле вызывать readdir() до NULL.

Для каждой записи, кроме . и .., при необходимости строить полный путь и вызывать lstat()/stat().

Закрыть каталог через closedir().

При этом readdir() не является потокобезопасной; для многопоточных программ используют readdir_r() (устарела) или внешнюю синхронизацию.

Размер каталога

Размер каталога, возвращаемый stat(), не равен сумме размеров файлов. Он зависит от количества записей и длины имён и определяется файловой системой.

Эти сведения лежат в основе утилиты ls -al и аналогичных программ, выполняющих обход каталога и вывод атрибутов его записей.

Исходный текст разработанной программы:

```
#include <sys/stat.h>
#include <sys/types.h>
#include <dirent.h>
#include <pwd.h>
#include <grp.h>
#include <time.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <limits.h> // Для PATH_MAX

void print_mode(mode_t m)
{
    char s[11] = "-----";

    // 1. Определяем тип файла (первый символ, как в ls -l)
    if (S_ISREG(m))
        s[0] = '-';
    else if (S_ISDIR(m))
        s[0] = 'd';
    else if (S_ISLNK(m))
        s[0] = 'l';
    else if (S_ISCHR(m))
        s[0] = 'c';
```

```

else if (S_ISBLK(m))
    s[0] = 'b';
else if (S_ISFIFO(m))
    s[0] = 'p';
else if (S_ISSOCK(m))
    s[0] = 's';

// 2. Определяем права доступа
if (m & S_IRUSR)
    s[1] = 'r';
if (m & S_IWUSR)
    s[2] = 'w';
if (m & S_IXUSR)
    s[3] = 'x';
if (m & S_IRGRP)
    s[4] = 'r';
if (m & S_IWGRP)
    s[5] = 'w';
if (m & S_IXGRP)
    s[6] = 'x';
if (m & S_IROTH)
    s[7] = 'r';
if (m & S_IWOTH)
    s[8] = 'w';
if (m & S_IXOTH)
    s[9] = 'x';

printf("%s", s);
}
int main(int argc, char **argv)
{
    // Если аргумент не передан, используем текущий каталог "."
    const char *dir = (argc > 1) ? argv[1] : ".";
    DIR *d = opendir(dir);
    if (!d)
    {
        perror("opendir");
        return 1;
    }
    struct dirent *e;
    char path[PATH_MAX];
    while ((e = readdir(d)) != NULL)
    {
        // Формируем полный путь для lstat
        snprintf(path, sizeof(path), "%s/%s", dir, e->d_name);
        struct stat st;
        // lstat считывает атрибуты самой ссылки, а не файла за ней
        if (lstat(path, &st) < 0)
        {
            perror("lstat");
            continue;
        }
    }
}

```

```

// Печатаем тип и права
print_mode(st.st_mode);
// Получаем имена владельца и группы (работа со статической памятью)
struct passwd *pw = getpwuid(st.st_uid);
struct group *gr = getgrgid(st.st_gid);
// Форматирование времени (Имитация поведения ls)
char tbuf[64];
time_t now = time(NULL);
// Если файл изменен более ~6 месяцев назад, ls показывает год, иначе время
if (difftime(now, st.st_mtime) > 60 * 60 * 24 * 182)
{
    strftime(tbuf, sizeof(tbuf), "%b %d %Y", localtime(&st.st_mtime));
}
else
{
    strftime(tbuf, sizeof(tbuf), "%b %d %H:%M", localtime(&st.st_mtime));
}
printf(" %ld %-8s %-8s %8ld %s %s\n",
        (long)st.st_nlink,
        pw ? pw->pw_name : "?",
        gr ? gr->gr_name : "?",
        (long)st.st_size,
        tbuf,
        e->d_name);
}

closedir(d);
return 0;
}

```

Результат:

```

boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$ ./2.3/2.3
drwxr-xr-x 2 boris boris 4096 Sep 20 12:23 4.1
drwxr-xr-x 2 boris boris 4096 Sep 20 12:25 4.2
drwxr-xr-x 2 boris boris 4096 Sep 20 11:09 2.1
drwxr-xr-x 2 boris boris 4096 Sep 20 11:41 3.2
drwxr-xr-x 2 boris boris 4096 Sep 20 12:04 3.6
drwxr-xr-x 2 boris boris 4096 Sep 20 11:31 2.3
drwxr-xr-x 2 boris boris 4096 Sep 20 11:14 2.2
drwxr-xr-x 2 boris boris 4096 Sep 20 12:06 3.7
drwxr-xr-x 8 boris boris 4096 Sep 20 12:28 .git
drwxr-xr-x 2 boris boris 4096 Sep 20 11:59 3.5
drwxr-xr-x 2 boris boris 4096 Sep 20 11:47 3.3
drwxr-xr-x 2 boris boris 4096 Sep 20 11:36 3.1
drwxr-xr-x 2 boris boris 4096 Sep 20 12:22 3.8
drwxr-xr-x 2 boris boris 4096 Sep 20 11:56 3.4
drwxr-x-- 19 boris boris 4096 Sep 20 10:50 ..
drwxr-xr-x 16 boris boris 4096 Sep 20 13:24 .
boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$

```

2. Применение неименованных каналов для связи между родственными процессами, исполняющими разные программы

В задании требуется реализовать применение неименованных каналов для связи между родственными процессами, исполняющими разные программы

Неименованный канал (anonymous pipe) — это средство межпроцессного взаимодействия в Unix/Linux, создаваемое системным вызовом `pipe(int fd[2])`. Канал существует только в ядре, не имеет имени в файловой системе и обычно используется между родственными процессами (родитель–потомок), так как дескрипторы наследуются через `fork()`.

Ключевые особенности

Два дескриптора: `fd[0]` открыт для чтения, `fd[1]` — для записи. Канал односторонний (half-duplex). Для двустороннего обмена нужно создавать два канала.

Поток байтов: границы сообщений не сохраняются. Читающий процесс может получить данные произвольными порциями; один `write()` не обязательно соответствует одному `read()`.

Блокировка: чтение из пустого канала блокируется до появления данных или закрытия всех пишущих концов. Запись в заполненный канал блокируется до освобождения места в буфере.

EOF: если все процессы закрыли `fd[1]`, то `read()` возвращает 0 (конец файла). Поэтому важно вовремя закрывать неиспользуемые концы канала.

SIGPIPE: если все читающие закрыли `fd[0]`, запись в канал вызывает сигнал SIGPIPE (или ошибку EPIPE, если сигнал игнорируется).

Атомарность: запись размером не более PIPE_BUF (POSIX, обычно ≥ 512 , в Linux часто 4096) гарантированно не перемешивается с другими записями. Более крупные записи могут чередоваться.

Буфер ограничен: объём буфера канала конечен (в Linux обычно 64 КБ), но полагаться на конкретное значение не следует.

Наследование: после `fork()` оба процесса получают копии дескрипторов, ссылающихся на один и тот же канал. Для корректной работы каждый процесс должен закрыть те концы, которые он не использует.

Неблокирующий режим: канал можно перевести в неблокирующий режим через `fcntl(fd, F_SETFL, O_NONBLOCK)`.

Только для родственных процессов: неименованный канал не может быть открыт по имени другим, неродственным процессом. Для связи произвольных процессов применяются именованные каналы (FIFO).

Типичное применение

Конвейеры командной оболочки (`cmd1 | cmd2`).

Передача данных от родительского процесса дочернему (и наоборот — через два канала).

Перенаправление стандартных потоков с помощью `dup2()`.

Таким образом, неименованный канал — простой, но ограниченный механизм: он удобен для однонаправленной передачи потока байтов между связанными процессами и требует аккуратного управления дескрипторами для предотвращения взаимных блокировок и утечек.

Исходный текст разработанной программы:

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc != 2)
        return 1;
    int fd = atoi(argv[1]);
    char buf[256];
    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    if (n > 0)
    {
        buf[n] = '\0';
        printf("Потомок прочитал: %s\n", buf);
    }
    close(fd);
    return 0;
}
```

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
int main()
{
    int fd[2];
    if (pipe(fd) < 0)
    {
        perror("pipe");
        exit(1);
    }

    pid_t pid = fork();
    if (pid < 0)
```

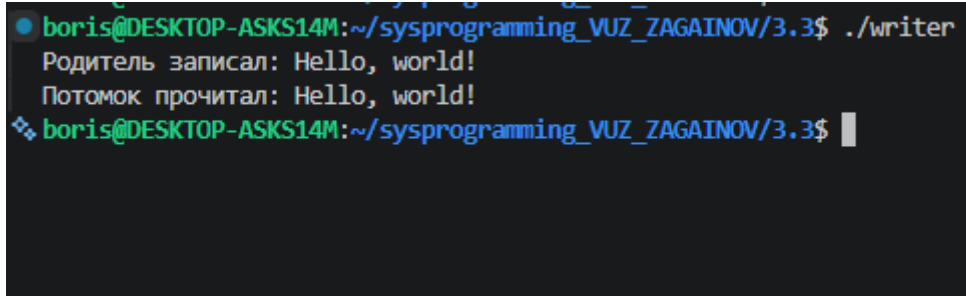
```

    {
        perror("fork");
        exit(1);
    }

    if (pid == 0)
    {
        close(fd[1]);
        char fdstr[16];
        sprintf(fdstr, "%d", fd[0]);
        execl("./reader", "reader", fdstr, NULL);
        perror("execl");
        exit(1);
    }
    else
    {
        close(fd[0]);
        char msg[] = "Hello, world!";
        write(fd[1], msg, sizeof(msg));
        printf("Родитель записал: %s\n", msg);
        close(fd[1]);
        wait(NULL);
    }
    return 0;
}

```

Результат:



```

boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV/3.3$ ./writer
Родитель записал: Hello, world!
Потомок прочитал: Hello, world!
❖ boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV/3.3$

```

3. Применение именованных каналов для связи между процессами

В задании требуется написать программное обеспечение с использованием именованных каналов для связи между ними.

Именованный канал (FIFO) — это специальный файл в файловой системе, который используется для однонаправленной передачи данных между процессами по принципу «первым вошёл — первым вышел». В отличие от неименованного канала, FIFO имеет имя (путь) и может быть открыт любыми, в том числе неродственными, процессами.

Создание и удаление

Создаётся системным вызовом `mkfifo(const char *pathname, mode_t mode)` или из командной строки: `mkfifo имя`.

Существует в файловой системе как специальный файл; права доступа задаются при создании с учётом `umask`.

После завершения работы FIFO обычно удаляют через `unlink(pathname)`, иначе он остаётся в системе.

Открытие и блокировки

Открывается обычным `open()` с флагами `O_RDONLY` или `O_WRONLY`.

`open()` только для чтения блокируется, пока какой-либо процесс не откроет FIFO для записи, и наоборот.

Флаг `O_NONBLOCK` снимает блокировку: чтение открывается сразу, запись — сразу, но при отсутствии читателей вернёт ошибку `ENXIO`.

Открытие `O_RDWR` нестандартно и не рекомендуется (поведение зависит от системы).

Передача данных

FIFO обеспечивает поток байтов: границы сообщений не сохраняются, один `write()` не обязан соответствовать одному `read()`.

Чтение из пустого FIFO блокируется до появления данных или закрытия всех пишущих дескрипторов.

Запись в заполненный FIFO блокируется до освобождения места в буфере.

Если все писатели закрыли FIFO, `read()` возвращает 0 (EOF).

Если все читатели закрыли FIFO, запись вызывает сигнал `SIGPIPE` (или ошибку `EPIPE`, если сигнал игнорируется).

Запись размером не более `PIPE_BUF` (POSIX ≥ 512 , в Linux обычно 4096) атомарна и не перемешивается с другими записями.

Буфер FIFO ограничен (в Linux обычно 64 КБ).

Особенности и применение

FIFO однонаправленный: для двустороннего обмена нужно создавать два канала.

Может использоваться между любыми процессами, а не только родственными.

Поддерживает `select()`, `poll()`, `epoll()` для мультиплексирования.

Данные не хранятся на диске — FIFO лишь точка встречи процессов.

Типичное применение: связь клиент–сервер, передача данных между независимыми утилитами, замена неименованных каналов там, где нужна гибкость.

Таким образом, именованный канал сочетает простоту `pipe` с возможностью адресации по имени, но требует аккуратного управления открытием/закрытием дескрипторов во избежание взаимных блокировок.

Исходный текст разработанной программы:

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc != 2)
    {
        fprintf(stderr, "Usage: %s fifo\n", argv[0]);
        return 1;
    }
    printf("Ожидание открытия FIFO на запись...\n");
    int fd = open(argv[1], O_WRONLY);
    if (fd < 0)
    {
        perror("open");
        return 1;
    }
    printf("FIFO открыт для чтения\n");
    char buf[256];
    ssize_t n = read(fd, buf, sizeof(buf) - 1);
    if (n > 0)
    {
        buf[n] = '\0';
        printf("Прочитано: %s\n", buf);
    }
    close(fd);
    return 0;
}

#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
```

```

#include <stdlib.h>
#include <errno.h>
int main(int argc, char **argv)
{
    if (argc != 2)
    {
        fprintf(stderr, "Usage: %s fifo\n", argv[0]);
        return 1;
    }
    // Создаем FIFO с проверкой ошибок
    if (mkfifo(argv[1], 0666) < 0)
    {
        // Если ошибка не "файл уже существует", то это критическая ошибка
        if (errno != EEXIST)
        {
            perror("mkfifo");
            return 1;
        }
        // Если файл уже существует, просто продолжаем работу
        printf("FIFO '%s' уже существует, используем его\n", argv[1]);
    }
    else
    {
        printf("FIFO '%s' успешно создан\n", argv[1]);
    }
    int fd = open(argv[1], O_WRONLY);
    if (fd < 0)
    {
        perror("open");
        return 1;
    }
    char msg[] = "Hello via FIFO!";
    write(fd, msg, sizeof(msg));
    printf("Записано в FIFO: %s\n", msg);
    close(fd);
    return 0;
}

```

Результат:

```

horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$ ./3.4/writer
Usage: ./3.4/writer fifo
horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$ ./3.4/writer fifo
FIFO 'fifo' успешно создан
Записано в FIFO: Hello via FIFO!
horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$

horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$ ./3.4/reader
Usage: ./3.4/reader fifo
horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$ ./3.4/reader fifo
Ожидание открытия FIFO на запись...
FIFO открыт для чтения
Прочитано: Hello via FIFO!
horis@DESKTOP-ASKS14H:~/sysprogramming_VUZ_ZAGANOV$

```

4. Мультиплексирование сообщений

В задании требуется разработать программное обеспечение с использованием мультиплексирования сообщений

Очереди сообщений

Очередь сообщений — механизм IPC, при котором процессы обмениваются структурированными сообщениями через буфер, управляемый ядром. В System V очередь идентифицируется ключом и дескриптором (`msgget`), а для работы используются `msgsnd`, `msgrcv`, `msgctl`.

Ключевые особенности

Сообщение состоит из типа (`long mtype, > 0`) и тела (`mtext`). Тип позволяет адресовать сообщения разным получателям или задавать приоритет.

Границы сообщений сохраняются: получатель читает ровно одно сообщение за вызов, а не поток байтов.

Выборка из очереди может быть:

первое сообщение любого типа (`mtype = 0`);

первое сообщение заданного типа (`mtype > 0`);

первое сообщение любого типа, кроме указанного (`mtype < 0`).

По умолчанию `msgsnd/msgrcv` блокируются, если очередь заполнена или пуста. Флаг `IPC_NOWAIT` делает вызов неблокирующим.

Очередь существует в ядре, не в файловой системе, и не удаляется автоматически при завершении процессов — требуется `msgctl(..., IPC_RMID, ...)`.

Есть системные ограничения: максимальный размер сообщения, общий объём очереди, число очередей.

Права доступа к очереди задаются аналогично файловым.

POSIX-очереди (`mq_open`, `mq_send`, `mq_receive`) предоставляют похожий интерфейс, но с приоритетами сообщений и уведомлениями.

Мультиплексирование сообщений

Мультиплексирование — организация обслуживания нескольких потоков сообщений одним процессом или каналом без создания отдельного процесса/потока на каждый источник.

В очередях сообщений

Одна очередь может обслуживать множество клиентов, если использовать разные типы сообщений для адресации. Сервер выбирает сообщения по типу (`msgrcv` с нужным

mtime) и тем самым демультиплексирует потоки от разных процессов. Это простой способ разделить логические каналы в одной очереди.

В сетевом взаимодействии

Сервер может одновременно ждать данные от нескольких сокетов с помощью select, poll или epoll. Эти вызовы принимают набор дескрипторов и возвращают те, что готовы к чтению/записи. Это позволяет одному процессу обслуживать много клиентов, не блокируясь на одном соединении. epoll масштабируется лучше при большом числе дескрипторов.

Общая идея

Мультиплексирование объединяет несколько логических потоков в один физический канал (или набор дескрипторов), а на принимающей стороне выполняется демультиплексирование — распределение сообщений по адресатам на основе типа, заголовка или дескриптора. Это уменьшает накладные расходы и упрощает архитектуру серверных приложений.

Исходный текст разработанной программы:

```
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/select.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SOCK_PATH "/tmp/chat.sock"

int main()
{
    int fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0)
    {
        perror("socket");
        exit(1);
    }

    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCK_PATH);

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
    {
        perror("connect");
        exit(1);
    }

    fd_set read_fds;
```

```

char buf[1024];
while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(0, &read_fds);
    FD_SET(fd, &read_fds);
    int maxfd = fd > 0 ? fd : 0;

    if (select(maxfd + 1, &read_fds, NULL, NULL, NULL) < 0)
    {
        perror("select");
        exit(1);
    }

    if (FD_ISSET(0, &read_fds))
    {
        int n = read(0, buf, sizeof(buf) - 1);
        if (n <= 0)
            break;
        send(fd, buf, n, 0);
    }

    if (FD_ISSET(fd, &read_fds))
    {
        int n = recv(fd, buf, sizeof(buf) - 1, 0);
        if (n <= 0)
            break;
        buf[n] = '\0';
        printf("Сообщение: %s", buf);
    }
}
close(fd);
return 0;
}

```

```

#include <sys/socket.h>
#include <sys/un.h>
#include <sys/select.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#define SOCK_PATH "/tmp/chat.sock"
#define MAX_CLIENTS 10

```

```

int main()
{
    int listen_fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (listen_fd < 0)
    {

```

```

    perror("socket");
    exit(1);
}

unlink(SOCK_PATH);
struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, SOCK_PATH);

if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
{
    perror("bind");
    exit(1);
}
if (listen(listen_fd, 5) < 0)
{
    perror("listen");
    exit(1);
}

int clients[MAX_CLIENTS];
for (int i = 0; i < MAX_CLIENTS; i++)
    clients[i] = -1;

fd_set master, read_fds;
FD_ZERO(&master);
FD_SET(listen_fd, &master);
int maxfd = listen_fd;

while (1)
{
    read_fds = master;
    if (select(maxfd + 1, &read_fds, NULL, NULL, NULL) < 0)
    {
        perror("select");
        exit(1);
    }

    if (FD_ISSET(listen_fd, &read_fds))
    {
        int client = accept(listen_fd, NULL, NULL);
        if (client >= 0)
        {
            for (int i = 0; i < MAX_CLIENTS; i++)
            {
                if (clients[i] == -1)
                {
                    clients[i] = client;
                    FD_SET(client, &master);
                    if (client > maxfd)
                        maxfd = client;
                }
            }
        }
    }
}

```

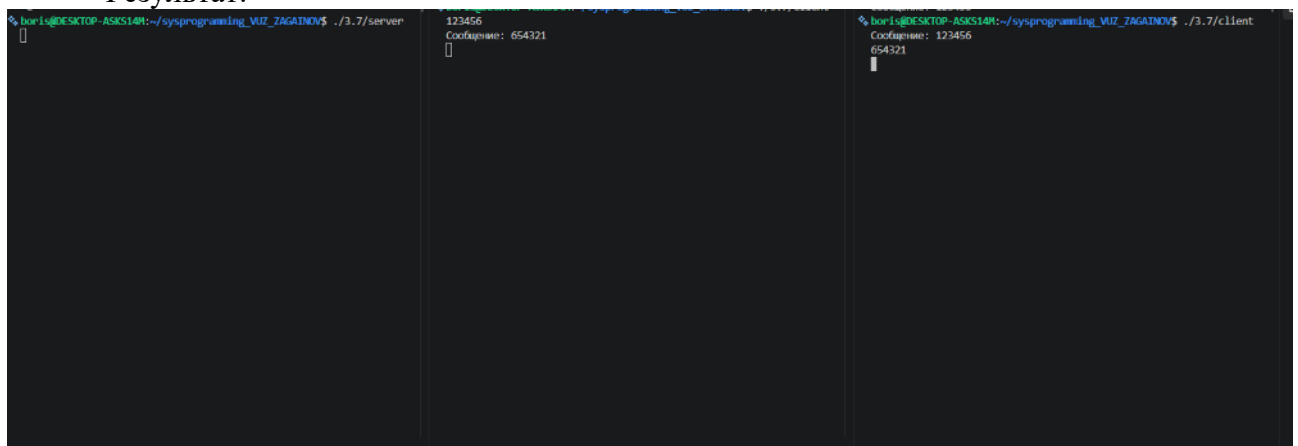
```

        break;
    }
}
}
}

for (int i = 0; i < MAX_CLIENTS; i++)
{
    int fd = clients[i];
    if (fd == -1)
        continue;
    if (FD_ISSET(fd, &read_fds))
    {
        char buf[1024];
        int n = recv(fd, buf, sizeof(buf) - 1, 0);
        if (n <= 0)
        {
            close(fd);
            FD_CLR(fd, &master);
            clients[i] = -1;
        }
        else
        {
            buf[n] = '\0';
            for (int j = 0; j < MAX_CLIENTS; j++)
            {
                if (clients[j] != -1 && clients[j] != fd)
                {
                    send(clients[j], buf, n, 0);
                }
            }
        }
    }
}
}
return 0;
}

```

Результат:



```

boris@DESKTOP-ASKS14R:~/sysprogramming_VUZ_ZAGAIKOV$ ./3.7/server
boris@DESKTOP-ASKS14R:~/sysprogramming_VUZ_ZAGAIKOV$ ./3.7/client

```

5. Многопоточность. Синхронизация потоков

В задании требуется разработать программное обеспечение, которое создаёт 2 дочерних потока и при наличии специального флага синхронизирует их

Исходный текст разработанной программы:

Многопоточное приложение и поток выполнения

Многопоточное приложение — программа, в которой одновременно выполняется несколько потоков (threads). Потоки существуют в рамках одного процесса и разделяют его адресное пространство: глобальные переменные, кучу, открытые файлы. При этом каждый поток имеет собственный стек, регистры и счётчик команд.

Поток выполнения — это последовательность инструкций, которая выполняется независимо от других потоков. Потоки могут работать параллельно (на многоядерных системах) или псевдопараллельно (с переключением контекста). Поскольку они используют общие данные, без координации возникают гонки (race conditions) и непредсказуемые результаты.

Цель синхронизации потоков

Синхронизация нужна, чтобы:

исключить одновременный доступ нескольких потоков к общим данным (взаимное исключение);

обеспечить согласованный порядок выполнения операций;

избежать состояний гонки, потери данных и взаимных блокировок;

организовать ожидание наступления события или завершения другого потока.

Механизм синхронизации, использованный в задании

В лабораторной работе 3.8 для опции --synchro применены мьютекс и условная переменная из библиотеки POSIX Threads:

`pthread_mutex_t mutex` — защищает общую переменную `turn`, которая указывает, чей сейчас ход.

`pthread_cond_t cond` — позволяет потоку ждать, пока не наступит его очередь.

Переменная `turn`: 1 — ход потока чисел, 2 — ход потока символов.

Алгоритм для каждого потока:

Захватить мьютекс.

Пока `turn` не равен «своему» значению — ждать на условной переменной (`pthread_cond_wait`).

Выполнить вывод своего элемента.

Сменить turn на значение другого потока.

Разбудить другой поток (pthread_cond_signal).

Освободить мьютекс.

Без --synchro потоки работают без синхронизации и выводят данные в произвольном порядке. Главный поток ожидает завершения обоих дочерних через pthread_join.

Такой механизм обеспечивает строго поочерёдный вывод: одно число — один символ, затем наоборот.

Результат:

```
boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$ ./3.8/3.8
0 1 2 3
4
```

```
boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$ ./3.8/3.8 --synchro
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
8
29
30
31
32
33      !
34      "
35      #
36      $
37      %
38      &
39      '
40      (
41      )
42      *
43      +
44      ,
45      -
46      .
47      /
```

6. Применение потоковых сокетов для организации взаимодействия процессов через сеть

В задании требуется реализовать применение потоковых сокетов для организации взаимодействия процессов через сеть

Потоковые сокет (SOCK_STREAM)

Потоковый сокет — это сокет, работающий по протоколу с установлением соединения (обычно TCP). Он обеспечивает надёжную, упорядоченную и двустороннюю передачу данных в виде потока байтов. Границы сообщений не сохраняются: один `send()` не обязан соответствовать одному `recv()`. Данные доставляются без потерь и дублирования; при необходимости TCP выполняет повторную передачу.

Основные свойства

Установление соединения перед обменом (трёхстороннее рукопожатие TCP).

Гарантированная доставка и порядок байтов.

Ориентация на поток, а не на сообщения.

Поддержка полного дуплекса (одновременно чтение и запись).

Адресация через IP-адрес и порт.

Порядок применения при взаимодействии сетевых приложений

Серверная сторона

`socket(AF_INET, SOCK_STREAM, 0)` — создать сокет.

Заполнить `struct sockaddr_in`: семейство `AF_INET`, адрес (`INADDR_ANY` или конкретный), порт (`htons(port)`).

`bind()` — привязать сокет к адресу и порту.

`listen()` — перевести сокет в режим прослушивания, задать очередь соединений.

`accept()` — принять входящее соединение; возвращает новый дескриптор для обмена с конкретным клиентом. Исходный сокет продолжает слушать.

Обмен данными с клиентом через `recv()/send()` или `read()/write()`.

`close()` — закрыть соединение.

Клиентская сторона

`socket(AF_INET, SOCK_STREAM, 0)` — создать сокет.

Заполнить `struct sockaddr_in` адресом и портом сервера (преобразование адреса — `inet_pton()`).

`connect()` — установить соединение с сервером.

Обмен данными через `send()/recv()` или `write()/read()`.

`close()` — закрыть соединение.

Особенности обмена

Для больших объёмов данных `send()/recv()` могут выполняться частично; нужно организовывать циклы.

Для одновременного обслуживания нескольких клиентов применяют `select()`, `poll()`, `epoll()` или многопоточность/многопроцессность.

Сервер обычно работает в бесконечном цикле: `accept()` → обработка → `close()`.

Корректное завершение: после `close()` одна сторона может получить 0 от `recv()` (EOF).

Таким образом, потоковые сокеты образуют основу большинства сетевых приложений (HTTP, FTP, SSH и др.), обеспечивая надёжный двусторонний обмен данными между клиентом и сервером.

Исходный текст разработанной программы:

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main()
{
    int s = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(12345);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    connect(s, (struct sockaddr *)&addr, sizeof(addr));

    char msg[] = "Hello TCP";
    send(s, msg, strlen(msg), 0);

    char buf[1024];
    int n = recv(s, buf, sizeof(buf) - 1, 0);
    if (n > 0)
    {
        buf[n] = '\0';
        printf("Ответ: %s\n", buf);
    }
    close(s);
    return 0;
}

#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
```

```

int main()
{
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

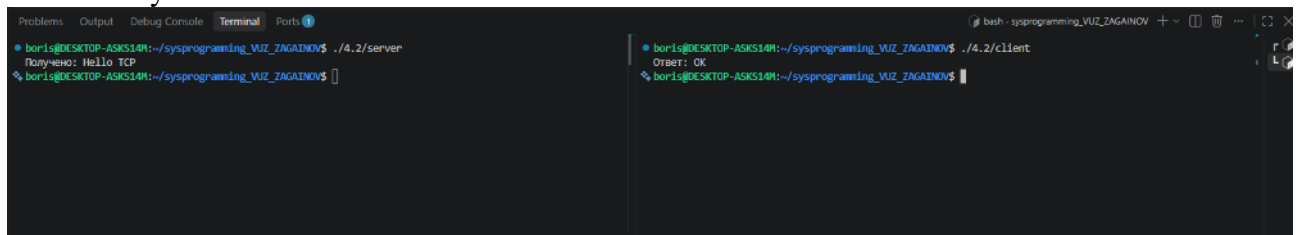
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(12345);

    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 5);

    int c = accept(s, NULL, NULL);
    char buf[1024];
    int n = recv(c, buf, sizeof(buf) - 1, 0);
    if (n > 0)
    {
        buf[n] = '\0';
        printf("Получено: %s\n", buf);
        send(c, "OK", 2, 0);
    }
    close(c);
    close(s);
    return 0;
}

```

Результат:



```

Problems  Output  Debug Console  Terminal  Ports
• boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$ ./4.2/server
Получено: Hello TCP
• boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$

• boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$ ./4.2/client
Ответ: OK
• boris@DESKTOP-ASKS14M:~/sysprogramming_VUZ_ZAGAINOV$

```

ЗАКЛЮЧЕНИЕ

...

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

...